Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота №6

із дисципліни «*Технології розроблення програмного забезпечення*»

Тема: «Патерни проектування»

Виконала:
Студент групи ІА-31
Самелюк А.С.

Перевірив:
Мягкий М.Ю.

Київ-2025

Тема: Патерни проектування.

Тема проєкту: Онлайн-щоденник (strategy, adapter, observer, facade, composite, client-server). Веб-застосунок дозволяє користувачу створювати, редагувати та організовувати особисті записи з можливістю пошуку, сортування та додавання тегів, а також отримання email-нагадувань.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

Посилання на репозиторій з проєктом та звітами:
https://github.com/Sameliuk/OnlineDiary_trpz/tree/main

https://github.com/Sameliuk/OnlineDiary_trpz/tree/reports

Хід роботи

1. Ознайомитись з короткими теоретичними відомостями.

   Шаблон «Abstract Factory»

   Призначений для створення сімейств пов'язаних об'єктів без зазначення їх конкретних класів.

   Використовується, коли потрібно забезпечити узгодженість елементів, які належать до одного стилю або типу.

   Приклад — створення об'єктів різних стилів у грі (стіни, двері, меблі).

   Переваги: узгодженість об'єктів, легке розширення, відокремлення створення від використання.

   Недоліки: складність коду та труднощі при додаванні нових типів продуктів.

   Шаблон «Factory Method»

   Визначає інтерфейс для створення об'єктів певного типу, дозволяючи підкласам вирішувати, який об'єкт створювати.

   Підходить, коли необхідно розширити систему новими типами без зміни існуючого коду.

Переваги: гнучкість, розширюваність, спрощене тестування.

Недоліки: може призвести до розростання ієрархій класів.

Шаблон «Memento» (Знімок)

Дозволяє зберігати і відновлювати стан об'єкта без порушення інкапсуляції. Стан зберігається у спеціальному об'єкті-знімку, який знає лише вихідний об'єкт.

Переваги: не порушує інкапсуляцію, спрощує реалізацію відкату дій.

Недоліки: збільшує використання пам'яті.

Шаблон «Observer» (Спостерігач)

Визначає залежність типу «один-до-багатьох», коли зміна стану одного об'єкта повідомляє всі підписані об'єкти.

Приклад — підписка на канал або повідомлення про зміну даних.

Переваги: слабке зв'язування об'єктів, динамічне додавання спостерігачів.

Недоліки: не визначається порядок сповіщення.

Шаблон «Decorator» (Декоратор)

Дозволяє динамічно додавати функціональність об'єктам без зміни їхнього коду. Декоратор «обгортає» базовий об'єкт і додає нову поведінку.

Переваги: більша гнучкість, можливість додавання поведінки під час виконання.

Недоліки: багато дрібних класів, складність при багаторівневому обгортанні.

2. Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

В даній частині було реалізовану логіку з нотатками та відредаговано логіку з нагадуваннями. Було реалізовано відповідні сервіси, контролери та інтерфейси для виконання певного функціоналу.

ReminderBack...ngService.cs | LogObserver.cs | ReminderService.cs | EmailObserver.cs | IReminderOl

OnlineDiaryApp

```
using OnlineDiaryApp.Observers;
using OnlineDiaryApp.Services;

2 references
public class ReminderBackgroundService : BackgroundService
{
    private readonly IServiceProvider _serviceProvider;

    0 references
    public ReminderBackgroundService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    0 references
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                using var scope = _serviceProvider.CreateScope();
                var reminderService = scope.ServiceProvider.GetRequiredService<ReminderService>();

                reminderService.Attach(scope.ServiceProvider.GetRequiredService<EmailObserver>());
                reminderService.Attach(scope.ServiceProvider.GetRequiredService<LogObserver>());

                var now = DateTime.Now;

                var reminders = (await reminderService.GetAllRemindersAsync())
                    .Where(r => r.Status == "active" && r.RemindAt <= now)
                    .ToList();

                foreach (var reminder in reminders)
                {
                    await reminderService.NotifyObserversAsync(reminder);

                    await reminderService.UpdateReminderAsync(reminder, newStatus: "sent");
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[Reminder service error]: {ex.Message}");
            }

            await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
        }
    }
}
```

Код 1 – ReminderBackgroundService (Services)

Create.cshtml* ReminderBackgroungService.cs LogObserver.cs Ema

OnlineDiaryApp

```
@model OnlineDiaryApp.Models.Note
@{
    ViewData["Title"] = "Створити нотатку";
}

<link rel="stylesheet" href="~/css/note.css" />

<div class="note-page">
    <div class="note-container">
        <h2>Створити нотатку</h2>

        <form asp-action="Create" method="post">
            <div class="form-group">
                <label>Заголовок</label>
                <input type="text" name="title" class="form-control" required />
            </div>

            <div class="form-group">
                <label>Текст</label>
                <textarea name="content" class="form-control" required></textarea>
            </div>


            <div class="form-group">
                <label for="tagSelect">Теги</label>

                <div id="tagSelectContainer">
                    <select id="tagSelect" class="form-control">
                        <option value="">— Оберіть тег —</option>
                        @foreach (var tag in ViewBag.Tags)
                        {
                            <option value="@tag.Id">@tag.Name</option>
                        }
                    </select>

                    <div id="selectedTags" class="selected-tags">
                        @if (Model != null && Model.Tags != null)
                        {
                            foreach (var tag in Model.Tags)
                            {
                                <span class="tag-item" data-id="@tag.Id">
                                    @tag.Name
                                    <button type="button" class="remove-tag">×</button>
                                </span>
                                <input type="hidden" name="tagIds" value="@tag.Id" />
                            }
                        }
                    </div>

                </div>
            </div>

            <div class="form-group">
                <label>Нагадування</label>
                <input type="datetime-local" name="reminderDate" class="form-control" />
            </div>

            <div class="note-actions">
                <a asp-action="Index" class="btn-cancel">Скасувати</a>
            <button type="submit" class="btn-save">Зберегти</button>
            </div>
        </form>
    </div>
</div>
```

Код 2.1 – Create (Views.Note)

```
<script>
    document.addEventListener('DOMContentLoaded', function () {
        const tagSelect = document.getElementById('tagSelect');
        const selectedTagsContainer = document.getElementById('selectedTags');

        tagSelect.addEventListener('change', function () {
            const selectedValue = this.value;
            const selectedText = this.options[this.selectedIndex].text;

            if (!selectedValue) return;

            if (selectedTagsContainer.querySelector(`[data-id="${selectedValue}"]`)) {
                this.value = "";
                return;
            }

            const tagEl = document.createElement('span');
            tagEl.className = 'tag-item';
            tagEl.dataset.id = selectedValue;
            tagEl.innerHTML = `${selectedText} <button type="button" class="remove-tag">×</button>`;

            const hiddenInput = document.createElement('input');
            hiddenInput.type = 'hidden';
            hiddenInput.name = 'tagIds';
            hiddenInput.value = selectedValue;

            selectedTagsContainer.appendChild(tagEl);
            selectedTagsContainer.appendChild(hiddenInput);

            this.value = "";
        });

        selectedTagsContainer.addEventListener('click', function (e) {
            if (e.target.classList.contains('remove-tag')) {
                const tagEl = e.target.closest('.tag-item');
                const tagId = tagEl.dataset.id;

                tagEl.remove();
                selectedTagsContainer
                    .querySelector(`input[value="${tagId}"]`)
                    ?.remove();
            }
        });
    });
</script>
```

Код 2.2 – Create (Views.Note)

Edit.cshtml | Create.cshtml* | ReminderBackgroungService.cs | LogObserver.cs | EmailObserver.cs

OnlineDiaryApp

```
@model OnlineDiaryApp.Models.Note

@{
    ViewData["Title"] = "Редагувати нотатку";
}

<link rel="stylesheet" href="~/css/note.css" />

<div class="note-page">
    <div class="note-container">
        <h2>Редагувати нотатку</h2>

        <form asp-action="Edit" method="post">
            <input type="hidden" asp-for="Id" />

            <div class="form-group">
                <label for="Title">Назва нотатки</label>
                <input asp-for="Title" class="form-control" />
                <span asp-validation-for="Title" class="text-danger"></span>
            </div>

            <div class="form-group">
                <label for="Content">Зміст нотатки</label>
                <textarea asp-for="Content" class="form-control"></textarea>
                <span asp-validation-for="Content" class="text-danger"></span>
            </div>

            <div class="form-group">
                <label for="tagSelect">Теги</label>

                <div id="tagSelectContainer">
                    <select id="tagSelect" class="form-control">
                        <option value="">— Оберіть тег —</option>
                        @foreach (var tag in ViewBag.Tags)
                        {
                            <option value="@tag.Id">@tag.Name</option>
                        }
                    </select>

                    <div id="selectedTags" class="selected-tags">
                        @foreach (var tag in Model.Tags)
                        {
                            <span class="tag-item" data-id="@tag.Id">
                                @tag.Name
                                <button type="button" class="remove-tag">×</button>
                            </span>
                            <input type="hidden" name="tagIds" value="@tag.Id" />
                        }
                    </div>
                </div>
            </div>

            <div class="form-group">
                <label for="remindAt">Нагадування (UTC)</label>
                <input type="datetime-local" id="remindAt" name="remindAt"
                       value="@(ViewBag.Reminder != null ? ((DateTime)ViewBag.Reminder.RemindAt).ToString("yyyy-MM-ddTHH:mm") : "")"
                       class="form-control" />
            </div>

            <div class="note-actions">
                <a asp-action="Index" class="btn-cancel">Скасувати</a>
                <button type="submit" class="btn-update">Оновити</button>
            </div>
        </form>
    </div>
</div>
```

Код 3.1 – Edit (Views.Note)

```
<script>
    document.addEventListener('DOMContentLoaded', function () {
        const tagSelect = document.getElementById('tagSelect');
        const selectedTagsContainer = document.getElementById('selectedTags');

        tagSelect.addEventListener('change', function () {
            const selectedValue = this.value;
            const selectedText = this.options[this.selectedIndex].text;

            if (!selectedValue) return;

            if (selectedTagsContainer.querySelector(`[data-id="${selectedValue}"]`)) {
                this.value = "";
                return;
            }

            const tagEl = document.createElement('span');
            tagEl.className = 'tag-item';
            tagEl.dataset.id = selectedValue;
            tagEl.innerHTML = `${selectedText} <button type="button" class="remove-tag">×</button>`;

            const hiddenInput = document.createElement('input');
            hiddenInput.type = 'hidden';
            hiddenInput.name = 'tagIds';
            hiddenInput.value = selectedValue;

            selectedTagsContainer.appendChild(tagEl);
            selectedTagsContainer.appendChild(hiddenInput);

            this.value = "";
        });

        selectedTagsContainer.addEventListener('click', function (e) {
            if (e.target.classList.contains('remove-tag')) {
                const tagEl = e.target.closest('.tag-item');
                const tagId = tagEl.dataset.id;

                tagEl.remove();
                selectedTagsContainer
                    .querySelector(`input[value="${tagId}"]`)
                    ?.remove();
            }
        });
    });
</script>
```

Код 3.2 – Edit (Views.Note)

Details.cshtml | Edit.cshtml* | Create.cshtml* | ReminderBackgroungService.cs | LogObser

OnlineDiaryApp

```
@model OnlineDiaryApp.Models.Note

<link rel="stylesheet" href="~/css/details.css" />

<div class="note-details">
    <h2>@Model.Title</h2>
    <p class="note-date">Створено: @Model.CreatedAt.ToString("f")</p>

    <div class="note-content">
        <h4> Зміст</h4>
        <p>@Model.Content</p>
    </div>

    <div class="note-tags">
        <h4> Теги:</h4>
        @if (Model.Tags.Any())
        {
            @foreach (var tag in Model.Tags)
            {
                <span class="note-tag">@tag.Name</span>
            }
        }
        else
        {
            <span class="note-tag-empty">Без тегів</span>
        }
    </div>

    @if (ViewBag.Reminder != null)
    {
        <div class="note-reminder">
            <h4> Нагадування:</h4>
            <p>@(((DateTime)ViewBag.Reminder.RemindAt).ToString("f"))</p>
        </div>
    }

    <div class="note-actions">
        <a asp-action="Edit" asp-route-id="@Model.Id" class="btn-edit">Редагувати</a>
        <a asp-action="Delete" asp-route-id="@Model.Id" class="btn-delete">Видалити</a>
        <a asp-action="Index" class="btn-back">Назад</a>
    </div>
</div>
```

Код 4 – Details (Views.Note)

NoteController.cs* | Details.cshtml | Edit.cshtml* | Create.cshtml* | ReminderBackgrou

OnlineDiaryApp

```
1   using Microsoft.AspNetCore.Mvc;
2   using OnlineDiaryApp.Models;
3   using OnlineDiaryApp.Services;
4
5   namespace OnlineDiaryApp.Controllers
6   {
        1 reference
7       public class NoteController : Controller
8       {
9           private readonly NoteService _noteService;
10          private readonly ReminderService _reminderService;
11          private readonly TagService _tagService;
12
            0 references
13          public NoteController(NoteService noteService, ReminderService reminderService, TagService tagService)
14          {
15              _noteService = noteService;
16              _reminderService = reminderService;
17              _tagService = tagService;
18          }
19
            0 references
20          public async Task<IActionResult> Index(string? sortBy, string? tag)
21          {
22              var userIdString = HttpContext.Session.GetString("UserId");
23              if (!int.TryParse(userIdString, out int userId))
24                  return RedirectToAction("Login", "User");
25
26              ISortStrategy? strategy = null;
27              if (!string.IsNullOrEmpty(sortBy))
28              {
29                  strategy = sortBy.ToLower() switch
30                  {
31                      "date" => new SortByDateStrategy(),
32                      "tag" when !string.IsNullOrEmpty(tag) => new SortByTagStrategy(tag),
33                      "title" => new SortByTitleStrategy(),
34                      _ => null
35                  };
36              }
37
38              var notes = await _noteService.GetAllNotesByUserAsync(userId, strategy);
39
40              ViewBag.SortBy = sortBy;
41              ViewBag.SelectedTag = tag;
42              ViewBag.Tags = await _tagService.GetAllTagsAsync(userId);
43
44              return View(notes);
45          }
46
            0 references
47          public async Task<IActionResult> Create()
48          {
49              var userIdString = HttpContext.Session.GetString("UserId");
50              if (!int.TryParse(userIdString, out int userId))
51                  return RedirectToAction("Login", "User");
52
53              var tags = await _tagService.GetAllTagsAsync(userId);
54              ViewBag.Tags = tags ?? new List<Tag>();
55
56              return View();
57          }
58
59          [HttpPost]
            0 references
60          public async Task<IActionResult> Create(string title, string content, List<int>? tagIds, DateTime? reminderDate)
61          {
62              var userIdString = HttpContext.Session.GetString("UserId");
63              if (!int.TryParse(userIdString, out int userId))
64                  return RedirectToAction("Login", "User");
65
66              var note = await _noteService.CreateNoteAsync(title, content, userId, tagIds ?? new List<int>());
67
68              if (reminderDate.HasValue)
69              {
70                  await _reminderService.CreateReminderAsync(note.Id, reminderDate.Value, userId);
71              }
72
73              return RedirectToAction("Index");
74          }
75
```

Код 5.1 – NoteController (Controllers)

```
76          0 references
            public async Task<IActionResult> Edit(int id)
77          {
78              var note = await _noteService.GetNoteByIdAsync(id);
79              if (note == null)
80                  return NotFound();
81
82              var userIdString = HttpContext.Session.GetString("UserId");
83              if (!int.TryParse(userIdString, out int userId))
84                  return RedirectToAction("Login", "User");
85
86              ViewBag.Tags = await _tagService.GetAllTagsAsync(userId) ?? new List<Tag>();
87              ViewBag.Reminder = await _reminderService.GetReminderByNoteIdAsync(id);
88
89              return View(note);
90          }
91
92          [HttpPost]
            0 references
93          public async Task<IActionResult> Edit(int id, string title, string content, List<int>? tagIds, DateTime? remindAt)
94          {
95              var note = await _noteService.GetNoteByIdAsync(id);
96              if (note == null)
97                  return NotFound();
98
99              note.Title = title;
100             note.Content = content;
101
102             await _noteService.UpdateNoteAsync(note, tagIds ?? new List<int>(), remindAt);
103
104             return RedirectToAction("Index");
105         }
106
            0 references
107         public async Task<IActionResult> Delete(int id)
108         {
109             var reminder = await _reminderService.GetReminderByNoteIdAsync(id);
110             if (reminder != null)
111             {
112                 await _reminderService.DeleteReminderAsync(reminder.Id);
113             }
114
115             await _noteService.DeleteNoteAsync(id);
116             return RedirectToAction("Index");
117         }
118
            0 references
119         public async Task<IActionResult> Search(string keyword)
120         {
121             var userIdString = HttpContext.Session.GetString("UserId");
122             if (!int.TryParse(userIdString, out int userId))
123                 return View(new List<Note>());
124
125             var notes = await _noteService.SearchByTitleAsync(keyword);
126             notes = notes.Where(n => n.UserId == userId);
127             return View("Index", notes);
128         }
129
            0 references
130         public async Task<IActionResult> Details(int id)
131         {
132             var note = await _noteService.GetNoteByIdAsync(id);
133             if (note == null)
134                 return NotFound();
135
136             ViewBag.Reminder = await _reminderService.GetReminderByNoteIdAsync(id);
137
138             return View(note);
139         }
140     }
141 }
142
```

Код 5.2 – NoteController (Controllers)

3. Реалізувати один з розглянутих шаблонів за обраною темою.

Для даного проєкту було обрано патерн Observer. Шаблон реалізований таким чином:

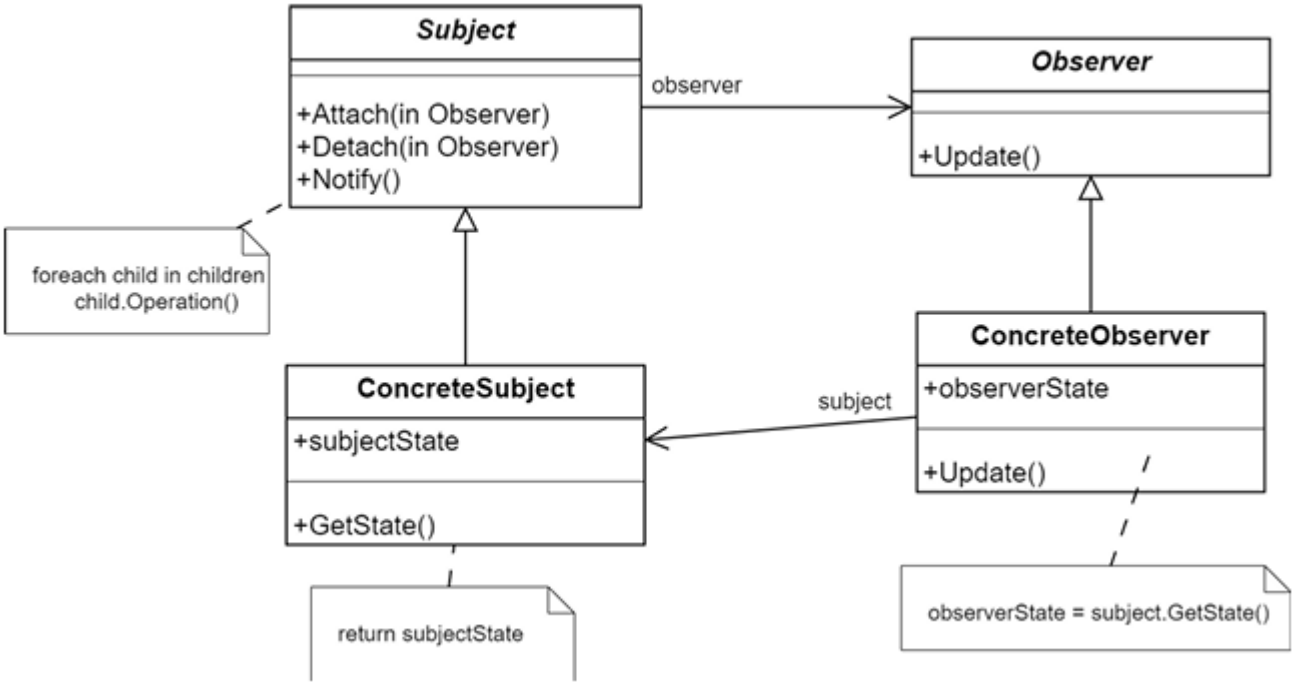


Рис.1 – Структура патерну Спостерігач

Subject: IReminderSubject – це інтерфейс, який визначає контракт для всіх суб'єктів спостереження. Він описує методи: Attach(IReminderObserver observer) – підключає нового спостерігача, Detach(IReminderObserver observer) – відключає спостерігача, NotifyObserversAsync(Reminder reminder, string action) – надсилає сповіщення всім спостерігачам про зміну стану. Завдяки інтерфейсу IReminderSubject можна створювати різні реалізації механізму сповіщення (наприклад, для нагадувань, подій, завдань тощо), не змінюючи код клієнтів, які з ним працюють.

ConcreteSubject: ReminderSubject – це конкретна реалізація інтерфейсу IReminderSubject. Він зберігає список об'єктів-спостерігачів (IReminderObserver) і відповідає за: додавання та видалення спостерігачів, розсилку повідомлень усім підписникам при зміні стану нагадування. Сам ReminderSubject не виконує бізнес-логіку нагадувань — він лише координує сповіщення спостерігачів про події ("created", "updated", "deleted", "time_reached").

Observer: IReminderObserver – це інтерфейс, який задає єдиний метод:
Task OnReminderChangedAsync(Reminder reminder, string action)
Цей метод викликається суб'єктом (ReminderSubject) у момент, коли змінюється стан нагадування. Кожен клас, що реалізує IReminderObserver, сам вирішує, як реагувати на ці зміни (логування, надсилання пошти, оновлення UI тощо).

ConcreteObservers: EmailObserver – реалізація спостерігача, який відправляє електронні листи користувачам, коли створюється, оновлюється або спрацьовує нагадування. Він використовує сервіс IEmailSender для відправлення повідомлень, але не знає, як саме це робиться (через SMTP, API тощо).
LogObserver – реалізація спостерігача, який записує інформаційні повідомлення про події нагадувань у консоль (журнал подій). Обидва спостерігачі підписуються на ті самі події, але реагують по-різному.

Client: ReminderService – це клієнт, який керує створенням, оновленням і видаленням нагадувань.
Він не реалізує власний механізм сповіщення, а використовує IReminderSubject для інформування зареєстрованих спостерігачів. Завдяки цьому ReminderService не залежить від конкретних дій, що виконуються при сповіщенні (наприклад, відправлення пошти чи логування). Він лише "говорить": "нагадування створено", а спостерігачі самі вирішують, що робити далі.

Завдяки патерну Observer система нагадувань стала гнучкою та розширюваною:

- можна легко додавати нові типи спостерігачів (наприклад, SMS-нагадування або push-повідомлення) без зміни коду ReminderService;
- різні модулі реагують на події незалежно один від одного;
- зберігається слабке зв'язування між компонентами — сервіс нагадувань не залежить від реалізації обробників подій.

Це і є суть патерну “Спостерігач” — встановлення механізму “підписки-сповіщення”, де об’єкти автоматично реагують на зміни стану іншого об’єкта без жорсткої залежності від нього.

4. Реалізувати не менше 3-х класів відповідно до обраної теми.

```
IReminderSubject.cs   ReminderService.cs   EmailObserver.cs   IReminderObserver.cs*   Details.cs
OnlineDiaryApp
1   using OnlineDiaryApp.Models;
2
3   namespace OnlineDiaryApp.Interfaces
4   {
        4 references
5       public interface IReminderSubject
6       {
            2 references
7           void Attach(IReminderObserver observer);
            2 references
8           void Detach(IReminderObserver observer);
            5 references
9           Task NotifyObserversAsync(Reminder reminder, string action);
10      }
11  }
12
```

Код 4 – IReminderSubject (Interfaces)

```
ReminderSubject.cs   IReminderSubject.cs   ReminderService.cs   EmailObserver.cs   IReminderObserver.cs*   Create.c
OnlineDiaryApp                                                                                  OnlineDiaryA
1   using OnlineDiaryApp.Interfaces;
2   using OnlineDiaryApp.Models;
3
4   namespace OnlineDiaryApp.Observers
5   {
        1 reference
6       public class ReminderSubject : IReminderSubject
7       {
8           private readonly List<IReminderObserver> _observers = new();
9
            2 references
10          public void Attach(IReminderObserver observer) => _observers.Add(observer);
            2 references
11          public void Detach(IReminderObserver observer) => _observers.Remove(observer);
12
            5 references
13          public async Task NotifyObserversAsync(Reminder reminder, string action)
14          {
15              foreach (var observer in _observers)
16                  await observer.OnReminderChangedAsync(reminder, action);
17          }
18      }
19  }
```

Код 5 – ReminderSubject (Observers)

```
IReminderObserver.cs*   Details.cshtml   Create.cshtml   Create.cshtml   Index.cshtml   tag-li
OnlineDiaryApp
1   using OnlineDiaryApp.Models;
2
3   namespace OnlineDiaryApp.Interfaces
4   {
        9 references
5       public interface IReminderObserver
6       {
            3 references
7           Task OnReminderChangedAsync(Reminder reminder, string action);
8       }
9   }
10
```

Код 6 – IReminderObserver (Interfaces)

```
EmailObserver.cs   IReminderObserver.cs*   Details.cshtml   Create.cshtml   Create.cshtml   Index.cshtml   tag-list.css   deta
OnlineDiaryApp                                                                OnlineDiaryApp.Observers.Email
1   using OnlineDiaryApp.Interfaces;
2   using OnlineDiaryApp.Models;
3
4   namespace OnlineDiaryApp.Observers
5   {
        3 references
6       public class EmailObserver : IReminderObserver
7       {
8           private readonly IEmailSender _emailSender;
9
            0 references
10          public EmailObserver(IEmailSender emailSender)
11          {
12              _emailSender = emailSender;
13          }
14
            2 references
15          public async Task OnReminderChangedAsync(Reminder reminder, string action)
16          {
17              if (action == "created" || action == "updated" || action == "time_reached")
18              {
19                  var user = reminder.User;
20                  if (user != null && !string.IsNullOrEmpty(user.Email))
21                  {
22                      await _emailSender.SendEmailAsync(
23                          user.Email,
24                          $"Нагадування: {reminder.Note?.Title ?? "Без назви"}",
25                          $"Ваше нагадування({action}).\n\n{reminder.Note?.Content ?? "Без тексту"}"
26                      );
27                  }
28              }
29          }
30      }
31  }
32
```

Код 7 – EmailObserver (Observers)

```
LogObserver.cs*   ReminderSubject.cs   IReminderSubject.cs   ReminderService.cs   EmailObserver.cs   IReminderObserver.cs*   Cre
OnlineDiaryApp                                                                    OnlineDiaryApp.Observers.LogOb
```

```
using OnlineDiaryApp.Interfaces;
using OnlineDiaryApp.Models;

namespace OnlineDiaryApp.Observers
{
    2 references
    public class LogObserver : IReminderObserver
    {
        2 references
        public Task OnReminderChangedAsync(Reminder reminder, string action)
        {
            Console.WriteLine($"[{DateTime.Now}] Reminder {action.ToUpper()} → ID={reminder.Id}," +
                $" NoteId={reminder.NoteId}, UserId={reminder.UserId}, Status={reminder.Status}");
            return Task.CompletedTask;
        }
    }
}
```

Код 8 – LogObserver (Observers)

```
ReminderService.cs   EmailObserver.cs   IReminderObserver.cs*   Details.cshtml   Creat
OnlineDiaryApp
```

```
using OnlineDiaryApp.Models;
using OnlineDiaryApp.Interfaces;
using OnlineDiaryApp.Repositories.Interfaces;

namespace OnlineDiaryApp.Services
{
    9 references
    public class ReminderService
    {
        private readonly IReminderRepository _reminderRepository;
        private readonly IUserRepository _userRepository;
        private readonly IReminderSubject _reminderSubject;

        0 references
        public ReminderService(
            IReminderRepository reminderRepository,
            IUserRepository userRepository,
            IReminderSubject reminderSubject)
        {
            _reminderRepository = reminderRepository;
            _userRepository = userRepository;
            _reminderSubject = reminderSubject;
        }

        2 references
        public void Attach(IReminderObserver observer) => _reminderSubject.Attach(observer);
        0 references
        public void Detach(IReminderObserver observer) => _reminderSubject.Detach(observer);

        3 references
        public async Task CreateReminderAsync(int noteId, DateTime remindAt, int userId)
        {
            var utcRemindAt = DateTime.SpecifyKind(remindAt, DateTimeKind.Utc);

            var reminder = new Reminder
            {
                NoteId = noteId,
                UserId = userId,
                RemindAt = utcRemindAt,
                Status = "active"
            };

            await _reminderRepository.AddAsync(reminder);
            await _reminderRepository.SaveChangesAsync();

            await _reminderSubject.NotifyObserversAsync(reminder, "created");
        }
```

Код 9.1 – ReminderService (Services)

```
2 references
public async Task UpdateReminderAsync(Reminder reminder, DateTime? newRemindAt = null, string? newStatus = null)
{
    if (newRemindAt.HasValue)
        reminder.RemindAt = DateTime.SpecifyKind(newRemindAt.Value, DateTimeKind.Utc);

    if (!string.IsNullOrEmpty(newStatus))
        reminder.Status = newStatus;

    await _reminderRepository.UpdateAsync(reminder);
    await _reminderRepository.SaveChangesAsync();

    await _reminderSubject.NotifyObserversAsync(reminder, "updated");
}

1 reference
public async Task<IEnumerable<Reminder>> GetAllRemindersAsync() =>
    await _reminderRepository.GetAllAsync();

4 references
public async Task<Reminder?> GetReminderByNoteIdAsync(int noteId) =>
    await _reminderRepository.GetByNoteIdAsync(noteId);

2 references
public async Task DeleteReminderAsync(int reminderId)
{
    var reminder = await _reminderRepository.GetByIdAsync(reminderId);
    if (reminder != null)
    {
        await _reminderRepository.DeleteAsync(reminder.Id);
        await _reminderRepository.SaveChangesAsync();

        await _reminderSubject.NotifyObserversAsync(reminder, "deleted");
    }
}

1 reference
public async Task NotifyObserversAsync(Reminder reminder)
{
    await _reminderSubject.NotifyObserversAsync(reminder, "time_reached");
}
}
}
```

Код 9.2 – ReminderService (Services)

5. Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Висновки: У ході лабораторної роботи було розглянуто та проаналізовано кілька шаблонів проєктування: «Абстрактна фабрика», «Фабричний метод», «Знімок» та «Декоратор». Було з'ясовано їх основне призначення, структуру та взаємодію між класами. Завдяки цим патернам можна підвищити гнучкість, масштабованість і підтримуваність програмного коду, розділити логіку створення об'єктів від їх використання, зберігати стан системи та розширювати функціональність без зміни базових класів. Отримані знання допомагають будувати більш структуровані та легко модифіковані програмні рішення.

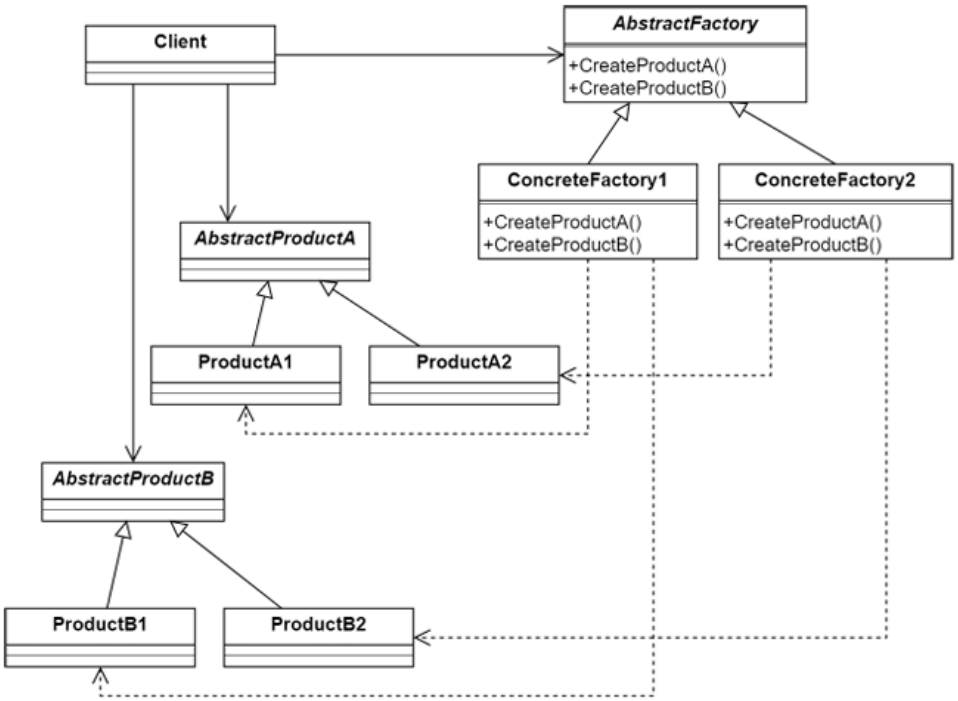
Відповіді на контрольні питання:

1. Призначення шаблону «Абстрактна фабрика»

Шаблон «Абстрактна фабрика» (Abstract Factory) призначений для створення сімейств пов'язаних об'єктів без прив'язки до конкретних класів.

Він дозволяє клієнту працювати лише з інтерфейсами, не знаючи, які саме об'єкти створюються.

2. Структура шаблону «Абстрактна фабрика»



3. Класи та їх взаємодія в «Абстрактній фабриці»

AbstractFactory – оголошує інтерфейс для створення абстрактних продуктів.

ConcreteFactory – реалізує методи створення конкретних продуктів.

AbstractProduct – описує інтерфейс продуктів.

ConcreteProduct – реалізує конкретний продукт.

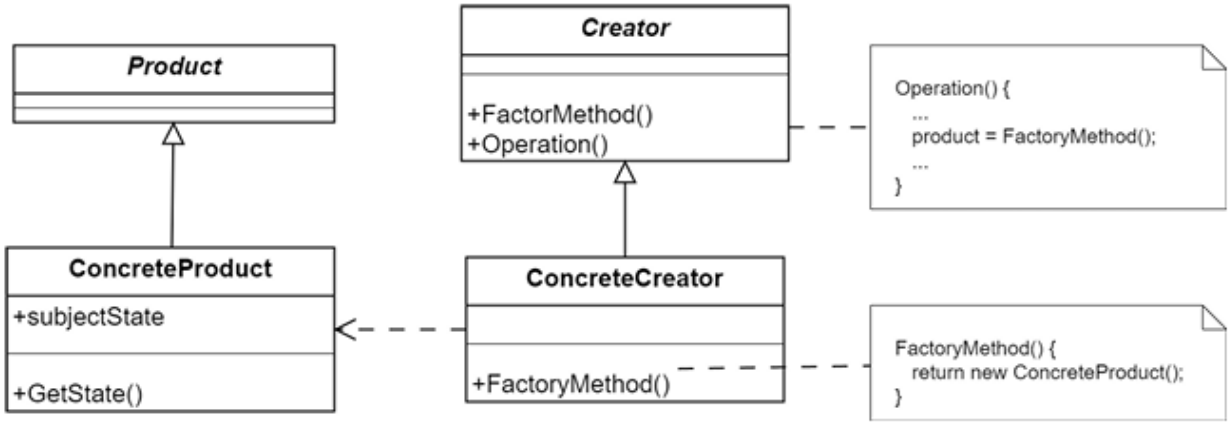
Client – працює лише з інтерфейсами AbstractFactory та AbstractProduct, не знаючи про їх реалізацію.

4. Призначення шаблону «Фабричний метод»

Шаблон **«Фабричний метод» (Factory Method)** визначає інтерфейс для створення об’єкта, але дозволяє підкласам вирішувати, який саме клас створювати.

Він делегує створення об’єктів підкласам.

5. Структура шаблону «Фабричний метод»



6. Класи та взаємодія у «Фабричному методі»

Product – спільний інтерфейс для всіх продуктів.

ConcreteProduct – конкретна реалізація продукту.

Creator – оголошує фабричний метод FactoryMethod().

ConcreteCreator – перевизначає FactoryMethod() для створення конкретного продукту.

Client – використовує об’єкт через базовий інтерфейс Product.

7. Відмінність між «Абстрактною фабрикою» та «Фабричним методом»

«Фабричний метод» визначає інтерфейс для створення одного типу об’єкта, але дозволяє підкласам вирішувати, який саме клас створювати. Тобто він зосереджується на створенні окремого продукту, надаючи механізм розширення без зміни існуючого коду. Основна ідея полягає в тому, щоб делегувати створення об’єкта підкласам.

«Абстрактна фабрика», своєю чергою, оперує на вищому рівні — вона створює цілі сімейства пов’язаних об’єктів, які повинні використовуватися

разом. Наприклад, фабрика може створювати набір інтерфейсних елементів (кнопки, меню, поля вводу), що належать до певного стилю (Windows, macOS тощо).

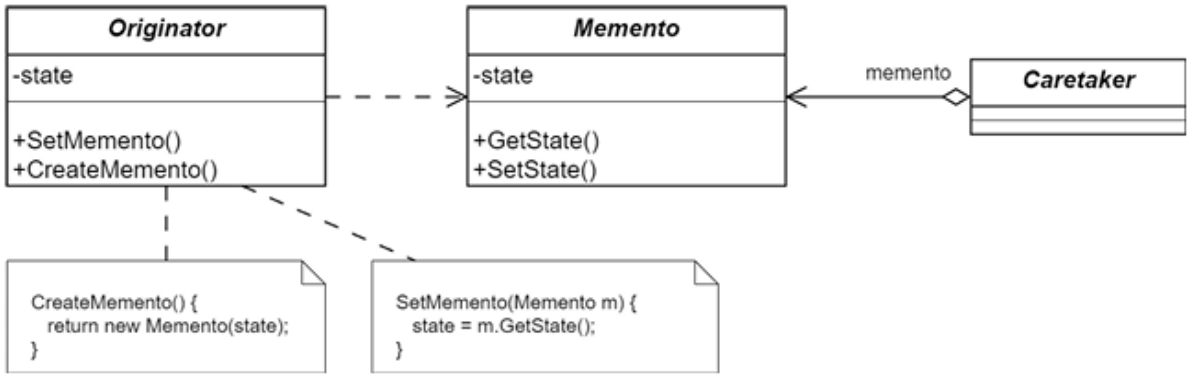
Отже, «Фабричний метод» вирішує, як створити один об'єкт, а «Абстрактна фабрика» — як створити набір сумісних **об'єктів**. Часто «Абстрактна фабрика» реалізується через кілька фабричних методів усередині себе.

8. Призначення шаблону «Знімок» (Memento)

Шаблон «Знімок» використовується для збереження і відновлення попереднього стану об'єкта без порушення інкапсуляції.

Він дозволяє "відкотити" об'єкт до попереднього стану (наприклад, у функції "Undo").

9. Структура шаблону «Знімок»



10. Класи та взаємодія у «Знімку»

Originator – об'єкт, стан якого потрібно зберегти. Створює Memento і відновлює стан із нього.

Memento – зберігає внутрішній стан Originator.

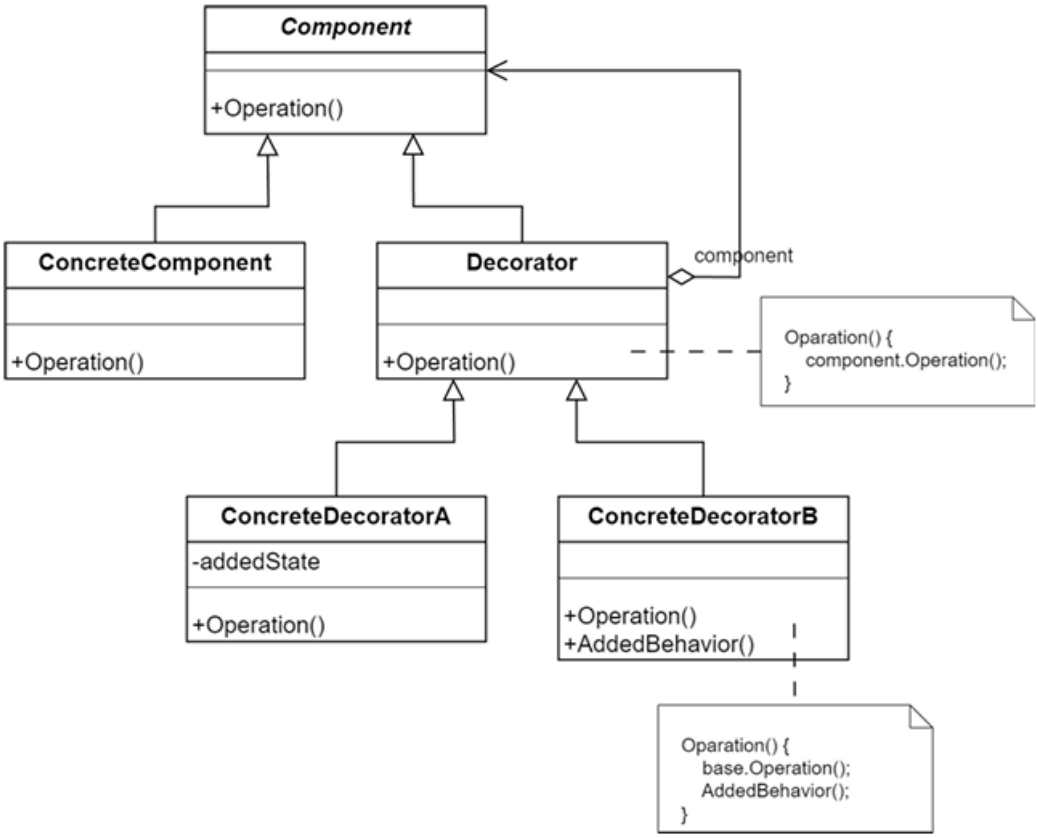
Caretaker – відповідає за збереження та відновлення Memento, але не змінює його вміст.

11. Призначення шаблону «Декоратор»

Шаблон «Декоратор» (Decorator) дозволяє динамічно додавати нову поведінку або функціональність об'єктам без зміни їх коду.

Він "обгортає" об'єкт у додатковий клас, який розширює його можливості.

12. Структура шаблону «Декоратор»



13. Класи та взаємодія у «Декораторі»

Component – базовий інтерфейс або клас.

ConcreteComponent – основний об'єкт, який може бути "обгорнутий".

Decorator – базовий клас, який реалізує інтерфейс Component і містить посилання на нього.

ConcreteDecorator – додає нову поведінку до об'єкта перед або після виклику базових методів.

14. Обмеження використання шаблону «Декоратор»

- Велика кількість маленьких об'єктів ускладнює відлагодження.

- Може бути важко зрозуміти послідовність викликів при вкладених декораторах.
- Не підходить, коли потрібно змінювати поведінку всіх об’єктів класу одразу (краще використати наслідування).